

AI Agent Servers and Startup Guide

Maitrova / Full-Stack-Project - Windows PowerShell

Prepared 18 September 2026. This guide explains what each service does, what must run, the exact startup commands, and how to check that the system is ready.

1. What needs to run?

Service	Purpose	Local port
MongoDB	Stores products, users, carts, orders, conversations, links, and agent jobs.	27017
Node / Express backend	Controls login, inventory, carts, checkout, payments, and orders.	5000
Python / FastAPI AI agent	Processes messages, calls Gemini, searches products, and handles WhatsApp jobs.	8000
React / Vite frontend	Displays the shopping website and authenticated checkout/linking pages.	5173
Cloudflare tunnel (optional)	Gives local FastAPI a public HTTPS address for Meta webhook testing.	Public HTTPS

Startup order

```
MongoDB -> Node backend -> FastAPI AI agent -> React frontend  
-> Cloudflare tunnel (only for local WhatsApp testing)
```

Keep three normal terminals open for Node, FastAPI, and React. Open a fourth for the tunnel when needed. MongoDB runs as a Windows service, so its terminal can be closed.

Important distinction

The Python virtual environment is not a server. It isolates the Python dependencies used by the AI server. Uvicorn is the server process that runs the FastAPI application.

Basis: project paths and commands discussed in this conversation, and the prior repository audit. No server secrets are included.

2. MongoDB and Node backend

MongoDB - the data store

The agent uses one logical database for its conversations and jobs, and a commerce database for store data. Both databases can live on the same MongoDB server. FastAPI and Node must point to the same commerce database.

Open PowerShell as Administrator and run:

```
Start-Service MongoDB
Get-Service MongoDB
```

Expected status: Running. If MongoDB is already running, skip Start-Service. MongoDB does not require a terminal to remain open.

If MongoDB is stopped, the application cannot connect and may show ECONNREFUSED 127.0.0.1:27017 or ::1:27017.

Node / Express - the commerce authority

This backend handles customer authentication, product and inventory rules, cart updates, orders, checkout, Razorpay callbacks, and WhatsApp order-status notifications. The AI agent calls Node for secure cart and order actions; it does not directly control payment state.

Open a normal PowerShell terminal:

```
cd C:\Users\home\Downloads\Full-Stack-Project\server
npm install
npm run dev
```

npm install is needed initially and when dependencies change. Keep npm run dev running. The default API base is <http://localhost:5000/api>.

Check the backend

```
http://localhost:5000/health
```

Configuration location: server/.env. Press Ctrl+C in the running terminal to stop Node.

3. Activate Python and run the agent

FastAPI - the agent processing layer

This service receives and stores customer messages, maintains context, calls Gemini for text/images, searches the live catalogue, builds replies, processes WhatsApp webhooks, and manages human handoffs. Its durable WhatsApp worker starts automatically with the server.

Open a new normal PowerShell terminal:

```
cd C:\Users\home\Downloads\Full-Stack-Project\server\ai-agent-backend
```

Create the virtual environment once

```
python -m venv .env
```

Activate it each time you open a new terminal

```
.\.env\Scripts\Activate.ps1
```

If PowerShell blocks activation, allow it for this terminal only:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass  
.\.env\Scripts\Activate.ps1
```

Install dependencies and start FastAPI

```
python -m pip install -r requirements.txt  
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Keep this terminal open. You do not need a separate command for the WhatsApp job worker. Configuration location: server/ai-agent-backend/.env.

Check the agent

```
http://localhost:8000/api/health  
http://localhost:8000/api/health/db  
http://localhost:8000/docs
```

Stop the server with Ctrl+C. Then leave the virtual environment with:

```
deactivate
```

4. Website and WhatsApp tunnel

React / Vite - the customer website

React renders the storefront. Vite serves it during development. Customers use it to browse, sign in, manage the cart, enter an address, pay, and complete secure WhatsApp account linking.

Open another normal PowerShell terminal:

```
cd C:\Users\home\Downloads\Full-Stack-Project\frontend
npm install
npm run dev
```

Open <http://localhost:5173> and keep the terminal running. Stop it with Ctrl+C.

Current website-chat limitation

The previous code audit found that React calls `/chat`, `/recommendations`, and `/chat/{session}`, but FastAPI exposes `/api/ai/chat` and has no matching `recommendation/history` routes. The chat launcher was also commented out. Starting the services alone does not repair those code issues; this guide does not claim they have since been fixed.

Cloudflare tunnel - local WhatsApp webhook access

Meta cannot call localhost on your computer. A tunnel exposes FastAPI through a public HTTPS address and forwards requests to port 8000. It is needed for local WhatsApp testing, not ordinary local website shopping.

With cloudflared installed, open another terminal:

```
cloudflared tunnel --url http://localhost:8000
```

Copy the generated HTTPS address into Meta webhook configuration, adding:

```
https://YOUR-TUNNEL-URL/api/whatsapp/webhook
```

Use the verification token configured as `WHATSAPP_VERIFY_TOKEN` in the agent `.env`. Keep the tunnel running. Temporary URLs may change after restart.

The webhook URL exposes only the agent. Customer checkout links and product image URLs must also be publicly reachable over HTTPS for real WhatsApp shopping.

Production difference

In production, use a permanent HTTPS domain and reverse proxy/ingress instead of a temporary tunnel. Build React and serve it with Nginx rather than Vite. Run Node, FastAPI, and MongoDB as managed services or separate containers.

5. Configuration, flow, and checks

External providers - not local servers

Provider	Purpose	Setup
Gemini	Message/image understanding and natural replies. Limited rule-based text fallback exists.	API key and model on FastAPI.
Meta WhatsApp Cloud API	Inbound messages and delivery of replies/images.	Token, phone ID, app secret, verify token, business ID, webhook.
Razorpay	Online payment processing and payment/refund callbacks.	Node payment credentials and webhook secret.

Required environment settings

- server/.env: MONGOOSE_URL, store authentication/payment settings, WhatsApp credentials, and WHATSAPP_COMMERCE_KEY.

- Agent .env: MONGODB_URL/MONGODB_DB_NAME, ECOMMERCE_MONGODB_URL/ECOMMERCE_MONGODB_DB_NAME, Gemini and WhatsApp settings, and public storefront/image URLs.

For local Node access, set in the agent .env:

```
ECOMMERCE_API_URL=http://127.0.0.1:5000/api
```

- WHATSAPP_COMMERCE_KEY must match in both backend .env files. Never expose server secrets through frontend/Vite variables.

- WHATSAPP_APP_SECRET verifies signed POST events. WHATSAPP_VERIFY_TOKEN verifies the GET challenge. They are different values.

How data moves

Website: Browser -> React -> Node -> MongoDB

WhatsApp: Customer -> Meta -> HTTPS tunnel -> FastAPI
-> MongoDB + Gemini -> Meta -> Customer

Purchase: Secure link -> React login -> Node validation
-> Cart -> Checkout -> Razorpay / eligible COD

Daily startup checklist

- MongoDB shows Running; Node /health and FastAPI /api/health/db respond successfully.

- React opens on port 5173. Check the terminal output if a different port is selected.

- For WhatsApp: verify the current HTTPS webhook, Meta credentials, public checkout origin, and public image URLs.

Troubleshooting

Port 27017 refused: start MongoDB. Port 5000 refused: start Node. Port 8000 refused: activate the environment, install requirements, and start Uvicorn. Missing Python module: run pip install using the active environment. HTTP 404 on browser chat: check endpoint alignment rather than starting duplicate servers.